

```

        // atomically because it's local data)
        Item item = ProduceItem();

        // wait until there's room
        WaitForSingleObject(&g_hFree);

        AddItemToQueue(&g_queue, item);

        // notify consumer that there's data
        SetEvent(&g_hUsed);
    }
    return nullptr;
}

void* ConsumerThreadEv(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        WaitForSingleObject(&g_hUsed);

        Item item = RemoveItemFromQueue(&g_queue);

        // notify producer that there's room
        SetEvent(&g_hFree);

        // consume the item (can be done non-
        // atomically because it's local data)
        ConsumeItem(item);
    }
    return nullptr;
}

void MainThread()
{
    // create event in the nonsignalled state
    g_hUsed = CreateEvent(nullptr, false,
                          false, nullptr);
    g_hFree = CreateEvent(nullptr, false,
                          true, nullptr);

    // spawn our threads
    CreateThread(nullptr, 0x2000, ConsumerThreadEv,
                0, 0, nullptr);
    CreateThread(nullptr, 0x2000, ProducerThreadEv,
                0, 0, nullptr);
}

```

```
    // ...  
}
```

4.7 Problems with Lock-Based Concurrency

In Section 4.5.3.2, we learned that *data races* can lead to incorrect program behavior in a concurrent system. We saw that the solution to this problem is to make operations on *shared* data objects *atomic*. One way to achieve atomicity is to wrap these operations in *locks*, which are often implemented using operating-system provided thread synchronization primitives such as mutexes.

However, atomicity is only part of the concurrency story. There are other problems that can plague a concurrent system, even when all shared-data operations have been carefully protected by locks. In the following sections, we'll briefly explore the most common of these problems.

4.7.1 Deadlock

Deadlock is a situation in which no thread in the system can make progress, resulting in a hang. When a deadlock occurs, all threads are in their Blocked states, waiting on some resource to become available. But because no threads are Runnable, none of those resources can ever become available, so the entire program hangs.

For a deadlock to occur, we need at least two threads and two resources. For example, Thread 1 holds Resource A but is waiting for Resource B; and at the same time, Thread 2 holds Resource B but is waiting for Resource A. This situation is illustrated by the following code snippet:

```
void Thread1()  
{  
    g_mutexA.lock(); // holds lock for Resource A  
    g_mutexB.lock(); // sleeps waiting for Resource B  
    // ...  
}  
  
void Thread2()  
{  
    g_mutexB.lock(); // holds lock for Resource B  
    g_mutexA.lock(); // sleeps waiting for Resource A  
    // ...  
}
```

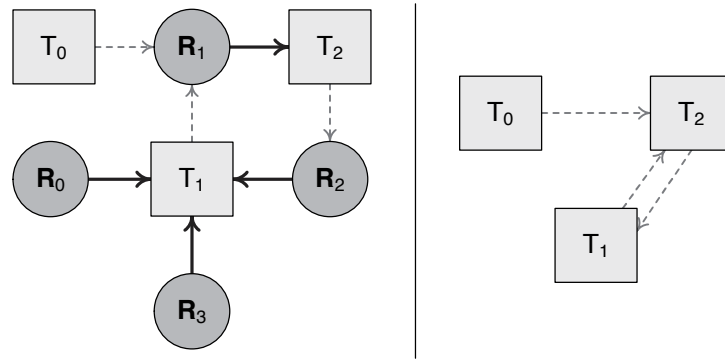


Figure 4.34. Left: Dark arrows show resources currently held by threads. Light dashed arrows show threads waiting on resources to become available. Right: We can eliminate the resources and simply draw the dependencies (light dashed arrows) between threads. A cycle in such a thread dependency graph indicates a deadlock.

Other more complex kinds of deadlock can of course occur as well, involving more threads and more resources. But the key factor that defines any deadlock situation is a *circular* dependency between threads and their resources.

To analyze a system for the possibility of deadlock, we can draw a graph of our threads, our resources, and the dependencies between them, as shown in Figure 4.34. In this graph, we've used squares to represent threads and circles to represent resources (or more precisely, the mutex locks that protect them). Solid arrows connect resources to the threads that currently hold locks on them. Dashed arrows connect threads to the resources they are waiting for. We can actually eliminate the resource nodes in the graph for simplicity if we like, leaving only the dashed lines connecting threads that are waiting on other threads. If such a dependency graph ever contains a *cycle*, we have a deadlock.

Actually a cycle in the dependency graph isn't quite enough to produce a deadlock. Strictly speaking, there are four necessary and sufficient conditions for deadlock, known as the *Coffman conditions*:

1. *Mutual exclusion*. A single thread can be granted exclusive access to a single resource via a mutex lock.
2. *Hold and wait*. A thread must be holding one lock when it goes to sleep waiting on another lock.
3. *No lock preemption*. No one (not even the kernel) is allowed to forcibly

break a lock held by a sleeping thread.

4. *Circular wait.* There must exist a cycle in the thread dependency graph.

Avoiding deadlock always boils down to preventing one or more of the Coffman conditions from holding true. Since violating conditions #1 and #3 would be “cheating,” solutions usually focus on avoiding conditions #2 and #4.

Hold and wait can be avoided by reducing the number of locks. In our simple example, if Resource A and Resource B were both protected by a single lock L, then deadlock could not occur. Either Thread 1 would obtain the lock, and gain exclusive access to both resources while Thread 2 waits, or Thread 2 would obtain the lock while Thread 1 waits.

The circular wait condition can be avoided by imposing a global order to all lock-taking in the system. In our simple two-threaded example, if we were to ensure that Resource A’s lock is always taken before Resource B’s lock, deadlock would be avoided. This would work because one thread will always obtain a lock on Resource A before attempting to take any other locks. This effectively puts all other contending threads to sleep, thereby ensuring that the attempt to take a lock on Resource B will always succeed.

4.7.2 Livelock

Another solution to the deadlock problem is for threads to *try* to take locks without going to sleep (using a function such as `pthread_mutex_trylock()`). If a lock cannot be obtained, the thread sleeps for some short period of time and then *retries* the lock.

When threads use an explicit strategy, such as timed retries, in order to avoid or resolve deadlocks, a new problem can arise: The threads can end up spending all of their time just trying to resolve the deadlock, rather than performing any real work. This is known as *livelock*.

As a simple example of livelock, consider again our example of two threads 1 and 2 contending over two resources A and B. Whenever a thread is unable to obtain a lock, it releases any locks it already holds and waits for a fixed timeout before trying again. If both threads use the same timeout, we can get into a situation in which the same degenerate situation simply repeats over and over. Our threads become “stuck” forever trying to resolve the conflict, and neither one ever gets a chance to do its real job. Livelock is akin to a stalemate in chess.

Livelock can be avoided by using an asymmetric deadlock resolution algorithm. For example, we could ensure that only one thread, either chosen at

random or based on priority, ever takes action to resolve a deadlock when one is detected.

4.7.3 Starvation

Starvation is defined as any situation in which one or more threads fail to receive any execution time on the CPU. Starvation can happen when one or more higher-priority threads fail to relinquish control of the CPU, thereby preventing lower-priority threads from running. Livelock is another kind of starvation, in which a deadlock-resolution algorithm effectively starves all threads of their ability to do “real” work.

Priority-based starvation is normally avoided by ensuring that high-priority threads never run for very long. Ideally a multithreaded program would consist of a pool of lower-priority threads that normally share the system’s CPU resources fairly. Once in a while, a higher-priority thread runs, takes care of its business quickly, and then ends, returning the CPU resources to the lower-priority threads.

4.7.4 Priority Inversion

Mutex locks can lead to a situation known as *priority inversion*, in which a low-priority thread acts as though it is the highest-priority thread in the system. Consider two threads, L and H, with a low and high priority, respectively. Thread L takes a mutex lock and then is preempted by H. If H attempts to take this same lock, then H will be put to sleep because L already holds the lock. This permits L to run even though it is lower priority than H—in violation of the principle that lower-priority threads should not run while a higher-priority thread is runnable.

Priority inversion can also occur if a medium-priority thread M preempts thread L while it is holding a lock needed by H. In this case, L goes to sleep while M runs, preventing it from releasing the lock. When H runs, it is therefore unable to obtain the lock; it goes to sleep, and now M’s priority has effectively been inverted with that of thread H.

The consequences of priority inversion may be negligible. For example, if the lower-priority thread immediately relinquishes the lock, the duration of the priority inversion may be short and it may go unnoticed or produce only minor ill effects. However, in extreme cases, priority inversion can lead to deadlock or other kinds of system failure. For example, a priority inversion can cause a high-priority thread to miss a critical deadline.

Solutions to the priority inversion problem include:

- Avoiding locks that can be taken by both low- and high-priority threads. (This solution is often not feasible.)
- Assigning a very high priority to the mutex itself. Any thread that takes the mutex has its priority temporarily raised to that of the mutex, thus ensuring it cannot be preempted while holding the lock.
- Random priority boosting. In this approach, threads that are actively holding locks are randomly boosted in priority until they exit their critical sections. This approach is used in the Windows scheduling model.

4.7.5 The Dining Philosophers

The famous *dining philosophers* problem is a great illustration of the problems of deadlock, livelock and starvation. It describes a situation in which five philosophers sit around a circular table, each with a plate of spaghetti in front of her. Between each philosopher sits a single chopstick. The philosophers wish to alternate between thinking (which they can do without any chopsticks), and eating (a task for which a philosopher requires two chopsticks). The problem is to define a pattern of behavior for the philosophers that ensures they can all alternate between thinking and eating without experiencing deadlock, livelock or starvation. (Obviously the philosophers represent threads, and the chopsticks represent mutex locks.)

You can read about this well-known problem online, so we won't devote a lot of space to discussing it here. However, it will be instructive to consider a few of the most common solutions to the problem:

- *Global order.* A dependency cycle can occur if a philosopher were always to pick up the chopstick to her left first (or always the one to her right). One solution to this problem is to assign each chopstick a unique global index. Whenever a philosopher wishes to eat, he or she always picks up the chopstick with the lowest index first. This prevents the dependency cycle, and hence avoids deadlock.
- *Central arbitor.* In this solution, a central arbitor (the "waiter") either grants a philosopher two chopsticks or none. This avoids the hold and wait problem by ensuring that no philosopher ever gets into a situation in which he or she holds only one chopstick, thereby preventing deadlock.
- *Chandra-Misra.* In this solution, chopsticks are marked either dirty or clean, and the philosophers send each other messages requesting chopsticks. You can read more about this solution by searching for "chandy-misra" online.

- $N - 1$ *philosophers*. For a table with N philosophers and N chopsticks, an integer semaphore can be used to limit the number of philosophers who are permitted to pick up chopsticks at any given time to $N - 1$. This solves the deadlock and livelock problems, because even in degenerate situations, at least one philosopher will succeed in obtaining two chopsticks. It does, however, permit one of the philosophers to starve, unless an additional “fairness” criterion is introduced.

4.8 Some Rules of Thumb for Concurrency

The solutions to the dining philosophers problem we described in the preceding section hint at some general principles and rules of thumb which we can apply to virtually any concurrent programming problem. Let’s take a brief look at a few of them.

4.8.1 Global Ordering Rules

In a concurrent program, the *order* in which events occur is not dictated by the order of the instructions in the program, as it is in a single-threaded program. If ordering is required in a concurrent system, that ordering must be imposed *globally*, across all threads.

This is one reason why a doubly linked list is not a concurrent data structure. A doubly linked list is designed the way it is in order to provide fast ($O(N)$) insertion and deletion of elements at any location within the list. Underlying this design is an assumption that *program order* is equivalent to *data order*—that when an ordered sequence of operations is performed on a list, the resulting list will contain correspondingly ordered elements. For example, let’s say we are given a linked list that contains the ordered elements { A, B, C }. If we execute the following two operations:

1. insert D before C,
2. insert E before C,

the assumption is that the resulting list will contain the ordered elements { A, B, D, E, C }.

In a single-threaded program, this assumption holds true. But in a multi-threaded system, program order no longer dictates data order. If one thread performs the insertion of D before C, and another thread performs the insertion of E before C, we have a race condition that could lead to any of the following results:

- { A, B, D, E, C },
- { A, B, E, D, C }, or
- { A, B, *corrupted data* }.

A corrupted list can occur if the data structure's operations aren't properly protected with critical sections. For example, both D's and E's "next" pointers might end up pointing to C.

A global ordering rule is the only viable solution to this problem. We first need to ask ourselves why the order of the elements in the list matters, and if it even matters at all. If order is unimportant, we can use a singly-linked list and always append the elements—an operation that can be implemented reliably either with locks or in a lock-free manner. And if a global ordering *is* required, we need to identify a stable and deterministic ordering criterion that does not depend on the order in which program events happen to occur. For example, we might sort the list alphabetically, or by priority, or some other useful criterion. The answers to these questions in turn dictate what kind of data structure we'll want to use. Attempting to use a doubly-linked list in a concurrent way (i.e., giving multiple threads mutable access to the list) is like trying to fit a square peg into a round hole.

4.8.2 Transaction-Based Algorithms

In the central arbiter solution to the dining philosophers problem, the arbiter or "waiter" hands out chopsticks in pairs: Either a philosopher receives all of the resources he needs (two chopsticks), or he receives none of them. This is known as a *transaction*.

A transaction can be more precisely defined as an indivisible bundle of resources and/or operations. Threads in a concurrent system submit transaction requests to a central arbiter of some kind. A transaction either succeeds in its entirety, or it fails in its entirety (because some other thread's transaction is actively being processed when the request arrives). If the transaction fails, its thread keeps resubmitting the transaction request until it succeeds (possibly waiting for a short time between retries).

Transaction-based algorithms are common in concurrent and distributed systems programming. And as we'll see in Section 4.9, the concept of a transaction underlies most *lock-free* data structures and algorithms.

4.8.3 Minimizing Contention

The most efficient concurrent system would be one in which all threads run without ever having to wait for a lock. This ideal can never be fully achieved,

of course, but concurrent systems programmers do attempt to minimize the amount of *contention* between threads.

As an example, consider a group of threads that are producing data and storing it into a central repository. Every time one of these threads attempts to store its data in the repository, it contends with all of the other threads for this shared resource. A simple solution that can sometimes work is to give each thread its own private repository. The threads can now produce data independently of one another, with no contention. When all of the threads are done producing their output, a central thread can collate the results.

The analog to this approach in the case of the dining philosophers problem would be to give each philosopher two chopsticks from the outset. Doing so would of course remove all concurrency from the problem—without any shared resources, there is no concurrency. In real-world concurrent systems we can't remove *all* sharing of resources, but we can certainly look for ways to *minimize* resource sharing in order to minimize lock contention.

4.8.4 Thread Safety

Generally speaking, a class or functional API is called *thread-safe* when its functions can be safely called by any thread in a multithreaded process. For any one function, thread safety is typically achieved by entering a critical section at the top of the function's body, performing some work, and then leaving the critical section just prior to returning. A class whose functions are all thread-safe is sometimes called a *monitor*. (The term "monitor" is also used to refer to a class that uses condition variables internally to permit clients to sleep while waiting for its protected resources to become available.)

Thread-safety is a convenient feature for a class or interface to offer. But it also imposes overhead that is sometimes unnecessary. For example, this overhead would be wasteful if the interface is being used in a single-threaded program, or if it's being used exclusively by one thread in a multithreaded program.

Thread-safety can also become a problem when an interface function needs to be called reentrantly. For example, if a class provides two thread-safe functions `A()` and `B()`, then these functions cannot call one another because each one independently enters and leaves the critical section. One solution to this problem is to use reentrant locks (see Section 4.9.7.3). Another is to implement "unsafe" versions of the functions in an interface, and then "wrap" each of them in thread-safe variant that simply enters a critical section, calls the "unsafe" function, and then leaves the critical section. That way, we can call the "unsafe" functions internally to the system, but the external interface of the

system remains thread-safe.

In my view, it's a bad idea to attempt to handle concurrent programming by simply creating classes and APIs that are 100% thread-safe. Doing so leads to interfaces that are unnecessarily heavy-weight, and it encourages programmers to ignore the fact that they are working in a concurrent environment. Instead, we should recognize and embrace the presence of concurrency in our software, and aim to design data structures and algorithms that address this concurrency explicitly. The goal should be to produce a software system that minimizes contention and dependencies between threads, and minimizes the use of locks. Achieving a completely lock-free system is a lot of work and is tough to get right. (See Section 4.9.) But striving in the *direction* of lock-freedom (i.e., minimizing the use of locks) is a far better strategy than over-using locks in a futile attempt to create “water-tight” interfaces that allow programmers to be oblivious of concurrency.

4.9 Lock-Free Concurrency

Thus far, all of our solutions to the problem of race conditions in a concurrent system have revolved around using *mutex locks* to make critical operations atomic, and possibly leveraging *condition variables* and the kernel's ability to *put threads to sleep* to avoid them wasting valuable CPU cycles in a busy-wait loop. During that discussion, we mentioned that there's another way to avoid race conditions that can potentially be more efficient. That approach is known as *lock-free* concurrency.

Contrary to popular belief, the term “lock-free” doesn't actually refer to the elimination of mutex locks, although that is certainly a component of the approach. In fact, “lock-free” refers to the practice of preventing threads from going to sleep while waiting on a resource to become available. In other words, in lock-free programming we never allow a thread to *block*. So perhaps the term “blocking-free” would have been more descriptive.

Lock-free programming is actually just one of a collection of *non-blocking* concurrent programming techniques. When a thread is blocked, it ceases to make progress. The goal of all of these techniques is to provide guarantees about the *progress* that can be made by the threads in the system, and by the system as a whole. We can organize these techniques into the following categories, listed in order of increasing strength of the progress guarantees they provide:

- *Blocking*. A blocking algorithm is one in which a thread can be put to sleep while waiting for shared resources to become available. A block-